

Java Arrays & Java Methods

A Concept Introduction

What You'll Learn

#	Topic	The question it answers
1	Creating an Array	How do I ask Java for one variable that can hold many values?
2	Initializing an Array	How do those many values actually get put in, and what's there before I put anything in?
3	Accessing an Array	How do I get one specific value back out, by position?
4	Accessing Using a for Loop	How do I reach every value, instead of typing out each index by hand?
5	Modifying an Array	What happens if I hand my array to a method — can it come back changed?
6	The length Property	How does an array know its own size, and how do I ask it?
7	2-Dimensional Arrays (Matrix)	What if one row of boxes isn't enough — what if I need a grid?
8	User-Defined Methods	How do I package a block of logic so I can use it by name, not by copy-paste?
9	Using Methods Multiple Times	What do I actually get back for writing a method once?
10	Making Methods More Dynamic	How does the same method produce different results for different situations?
11	Java Static Methods	Why does main() not need an object to run — and when should my own methods follow suit?
12	Java Standard Library Methods	How much of this have other people already solved for me?

SECTION 1

Creating an Array

One variable, many boxes

A mentor tracking five students' quiz scores could write five separate variables — `score1`, `score2`, `score3`, `score4`, `score5` — and it would work, right up until the class grows to fifty students, or the mentor needs to loop through every score to find the average. At that point, five (or fifty) separate variables stop being a minor inconvenience and become the actual obstacle.

An array is Java's answer: one single variable name that holds many values of the same type, each one reachable by its position.

```
int[] quizScores;           // declaring an array of int -- no boxes exist yet
quizScores = new int[5];    // now Java reserves space for exactly 5 int values

// or in one line:
int[] quizScores = new int[5];

// arrays work for any type, not just int:
String[] studentNames = new String[30];
double[] cgpaValues = new double[30];
```

WHY THIS MATTERS

`new int[5]` tells Java exactly how many boxes to reserve, up front, in one contiguous block of memory — which is precisely what makes reaching any one of them by position so fast.

The size you ask for is a promise: five boxes exist and no more. Anything that needs to grow past that size needs a bigger array, not a bigger box.

PAUSE & THINK

`quizScores.length` is fixed the moment `new int[5]` runs. If a sixth student joins the class, what do you think actually has to happen to the array — does Java quietly add a sixth box, or is there no such thing as “adding a box” to an existing array at all?

Two Ways to Write the Type

```
int[] scores = new int[5];    // preferred style -- the [] sits with the type
int scores[] = new int[5];    // also legal Java, inherited from C-style syntax -- avoid it
```

WATCH OUT FOR

Forgetting the size entirely — `int[] scores = new int[];` does not compile. Java needs to know how many boxes to reserve at the moment of creation.

Assuming an array can be resized later the way some other languages allow — in Java, a fixed-size array simply cannot grow or shrink after creation.

QUICK RECAP

An array is one variable name holding many values of the same type, each reachable by position.

`new int[5]` both reserves the memory and fixes the size, permanently, for that array object.

SECTION 2

Initializing an Array

What's already in the boxes, and how to fill them yourself

`new int[5]` doesn't leave five boxes empty — Java fills every one of them with a default value immediately, the same defaults you already met with class fields: 0 for numeric types, false for boolean, null for any object type like String.

```
int[] scores = new int[5];
System.out.println(scores[0]); // 0 -- not garbage, not blank. exactly 0.

boolean[] attended = new boolean[5];
System.out.println(attended[0]); // false

String[] names = new String[5];
System.out.println(names[0]); // null -- no String object exists in this box yet
```

PAUSE & THINK

If you print `names[0]` before assigning anything to it and get null, and then you try `names[0].length()`, what do you think happens? Think about what null actually means before answering.

Filling an Array With Known Values Up Front

When you already know the values at the moment you're writing the code, array-literal syntax skips the default-filling step entirely and lets Java size the array to match.

```
int[] quizScores = {85, 90, 78, 92, 88}; // size is automatically 5 -- inferred from the list
String[] toppers = {"Ravi", "Meera", "Karthik"};
```

WHY THIS MATTERS

Java's default-value guarantee is what makes patterns like “start a counter array at all zeros and add to it as you go” completely safe — you never have to manually zero out an array before using it as an accumulator.

WATCH OUT FOR

Mixing the two initialization styles incorrectly: `int[] scores = new int[5]{85,90,78,92,88};` does not compile. Use `new int[5]` alone (defaults), or `{85,90,78,92,88}` alone (literal) — never both together.

QUICK RECAP

`new int[5]` fills every box with that type's default value — 0, false, or null.

A literal list `{85, 90, 78}` both creates the array and fills it in one step, with its size inferred from how many values you listed.

SECTION 3

Accessing an Array

Reaching one specific box, by position

Every box in an array has a position number, called its index — and Java, like most languages, starts counting from 0, not 1. The first score is at index 0, the second at index 1, and so on.

```
int[] quizScores = {85, 90, 78, 92, 88};  
//           0   1   2   3   4   <- these are the indices  
  
System.out.println(quizScores[0]); // 85 -- the FIRST score  
System.out.println(quizScores[3]); // 92 -- the FOURTH score, at index 3
```

PAUSE & THINK

quizScores has 5 elements. What's the index of the very last one — is it 5, or something else? Work it out before checking the next line.

The last valid index is always length – 1. For a 5-element array, that's index 4 — quizScores[4] is the last score, and quizScores[5] does not exist.

```
System.out.println(quizScores[5]);  
// ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
```

WHY THIS MATTERS

This exception exists to protect you: silently reading or writing memory past the end of an array is exactly the class of bug that used to cause serious security vulnerabilities in languages that allow it. Java refuses, loudly, the instant it happens.

WATCH OUT FOR

The classic off-by-one trap: for an array of length n , valid indices run from 0 to $n-1$ — not 1 to n . This single miscount is one of the most common beginner bugs in any language.

Using a negative index, like quizScores[-1] to try to grab “the last element” the way some other languages allow — Java has no such shortcut; a negative index throws the same exception as one that's too large.

QUICK RECAP

Indexing starts at 0. The last valid index of an array is always length – 1.

Reading or writing outside that range throws ArrayIndexOutOfBoundsException immediately — Java never lets it pass silently.

SECTION 4

Accessing Using a for Loop

Reaching every box without typing every index by hand

quizScores[0], quizScores[1], quizScores[2]... typed out by hand works for five students. It does not work for fifty, and it definitely does not work when you don't know the array's size while you're writing the code. A for loop, combined with the array's own length, solves both problems at once.

```
int[] quizScores = {85, 90, 78, 92, 88};

for (int i = 0; i < quizScores.length; i++) {
    System.out.println("Score " + i + ": " + quizScores[i]);
}
```

The Enhanced for Loop (for-each)

When you only need each value, and don't care about its index at all, Java offers a shorter form that reads almost like plain English.

```
for (int score : quizScores) {
    System.out.println("Score: " + score);
}
```

PAUSE & THINK

The for-each loop above never mentions an index at all. If you needed to print “Student #3 scored 92” — the position AND the value together — could you still use for-each, or do you need the index-based loop instead?

WHY THIS MATTERS

Looping with `i < quizScores.length` instead of a hardcoded `i < 5` means the exact same loop keeps working correctly even if the array's size changes later — one less thing to remember to update.

WATCH OUT FOR

Using the for-each loop when you actually need to know each element's position (to print it, compare it to a neighbor, or modify it) — for-each gives you the value only, never the index.

QUICK RECAP

A standard for loop with `i < array.length` reaches every index, and gives you the index itself when you need it.

A for-each loop (`for (type x : array)`) is shorter when you only need each value, never its position.

SECTION 5

Modifying an Array

What happens when you hand your array to someone else

Changing one element by index is the easy part — `quizScores[2] = 95`; simply overwrites whatever was there. The genuinely interesting question is what happens when an array is passed into a method. Back in Week 3, you learned Java is strictly pass-by-value. So does a method that modifies an array parameter actually change the caller's original array, or not?

```
public static void curveScores(int[] scores) {
    for (int i = 0; i < scores.length; i++) {
        scores[i] = scores[i] + 5;    // add 5 bonus marks to every score
    }
}

public static void main(String[] args) {
    int[] quizScores = {85, 90, 78, 92, 88};
    curveScores(quizScores);
    System.out.println(quizScores[0]);    // what prints here?
}
```

PAUSE & THINK

Predict the output of that last line before reading on. Does `curveScores()` change the original `quizScores` array in `main()`, or does it only change its own local copy?

It prints 90 — the original array really did change. This isn't a contradiction of pass-by-value: what gets copied into `scores` is the array's reference (its address), not the array's contents. Both `quizScores` in `main()` and `scores` inside `curveScores()` end up pointing at the exact same set of boxes in memory.

WHY THIS MATTERS

This is exactly why a method like `Arrays.sort(myArray)` can sort your array in place, with no return value at all — it's modifying the same boxes you already have, through the reference it was handed.

It also means a method that isn't supposed to modify your array can still do so by accident, unless it deliberately copies the array first — the same reference-sharing risk you saw with mutable fields back in Week 5.

WATCH OUT FOR

Assuming an array is somehow protected from modification just because it was only passed as a method parameter — unlike an `int` or a `double`, an array parameter gives the method direct access to the very same boxes the caller has.

QUICK RECAP

Reassigning one element: `array[index] = newValue`; overwrites that box directly.

Arrays are passed to methods by reference-copy: the method can see and modify the caller's actual array, not a separate copy of its contents.

SECTION 6

Using the length Property of an Array

One word, no parentheses — and a very common mix-up

Every array carries its own size with it, available the instant you need it, through `length`. You've already used it several times in this document without a second thought — this section is about the one detail that trips up almost everyone the first time.

```
int[] quizScores = {85, 90, 78, 92, 88};
System.out.println(quizScores.length);    // 5 -- no parentheses

String name = "Ravi";
System.out.println(name.length());        // 4 -- WITH parentheses
```

PAUSE & THINK

`quizScores.length` and `name.length()` look almost identical, but one has parentheses and one doesn't. Before reading on: which one is a method, and which one is a field — and does that distinction match what you already know about the difference between the two?

`length` on an array is a field — a plain piece of data attached to the array object, read directly with no parentheses. `length()` on a String is a method — a piece of behavior you call, hence the parentheses. Arrays and Strings simply chose different conventions here, and Java gives you no warning if you mix them up in the wrong direction; it's simply a compile error to get it backwards.

WATCH OUT FOR

Writing `quizScores.length()` with parentheses — arrays do not have a `length()` method, only a `length` field. This is a compile error, not a runtime surprise.

Writing `name.length` without parentheses on a String — the opposite mistake, equally common, equally a compile error.

QUICK RECAP

`array.length` — no parentheses, it's a field.

`string.length()` — parentheses required, it's a method.

Use `array.length` instead of a hardcoded number in loop conditions, so the loop stays correct even if the array's size changes.

SECTION 7

Java 2-Dimensional Arrays (Matrix)

What if one row of boxes isn't the shape your data actually has?

A single `quizScores` array works when you're tracking one quiz for one class. But a mentor tracking every student's score across every quiz of the semester has data that's naturally a grid — rows of students, columns of quizzes — not a single line of boxes.

```
int[][] quizGrid = new int[3][4];    // 3 students, 4 quizzes each -- 12 boxes total, arranged in a
grid

quizGrid[0][0] = 85;    // student 0's score on quiz 0
quizGrid[0][1] = 90;    // student 0's score on quiz 1
quizGrid[1][0] = 78;    // student 1's score on quiz 0
```

A 2D array is really an array of arrays: `quizGrid` is an array where every single element is itself an `int[]`. Literal initialization makes that structure visible directly in the code:

```
int[][] quizGrid = {
    {85, 90, 78, 92},    // student 0's four quiz scores
    {70, 88, 95, 80},    // student 1's four quiz scores
    {60, 75, 82, 91},    // student 2's four quiz scores
};
```

Traversing a Grid With Nested Loops

```
for (int student = 0; student < quizGrid.length; student++) {
    for (int quiz = 0; quiz < quizGrid[student].length; quiz++) {
        System.out.print(quizGrid[student][quiz] + "\t");
    }
    System.out.println();    // move to the next row after each student
}
```

PAUSE & THINK

The inner loop's condition is `quiz < quizGrid[student].length`, not `quiz < quizGrid[0].length` or a hardcoded `quiz < 4`. Can you think of a legitimate situation where two different students might genuinely have a different number of quiz scores recorded — and why the code above already handles that correctly?

WHY THIS MATTERS

`quizGrid.length` gives you the number of rows (students); `quizGrid[student].length` gives you the number of columns in that specific row (that student's quiz count) — and in Java, those two numbers don't have to match across every row. A grid where rows have different lengths is called a jagged array, and it's completely legal.

Real tabular data — seating charts, spreadsheets, game boards, pixel grids — is naturally two-dimensional, and 2D arrays let your code's structure mirror the data's actual shape.

WATCH OUT FOR

Assuming every row of a 2D array must be the same length just because most examples show a neat rectangle — Java only guarantees that `quizGrid` is an array of `int[]` references; nothing forces those inner arrays to all be the same size.

QUICK RECAP

`int[][]` declares an array of arrays; `quizGrid[row][col]` reaches one specific box.

`quizGrid.length` is the number of rows; `quizGrid[row].length` is that specific row's own length — rows are not required to match.

Nested for loops (outer for rows, inner for columns) are the standard way to visit every box in a grid.

SECTION 8

User-Defined Methods

Packaging a block of logic so you can use it by name

Computing the average of a section's quiz scores is a handful of lines: loop through the array, add everything up, divide by the count. Needing that same average for five different sections means either copy-pasting those lines five times, or writing them once, giving them a name, and calling that name whenever it's needed. A method is exactly that: a named, reusable block of code.

```
public static double calculateAverage(int[] scores) {
    int total = 0;
    for (int score : scores) {
        total += score;
    }
    return (double) total / scores.length;
}
```

The Anatomy of a Method

Part	In the Example Above	Meaning
Return type	double	What kind of value this method hands back when it's done.
Name	calculateAverage	How you'll refer to this method everywhere else in your code.
Parameters	(int[] scores)	What information the method needs from whoever calls it.
Body	{ ... }	The actual logic — written once, reused forever.
return	return (double) total / scores.length;	Sends a value back to whoever called the method, and ends the method immediately.

SECTION 9

WHY THIS MATTERS

This is the same “don't repeat yourself” instinct you've already met with constructors and `this()` chaining — methods are the general-purpose version of that idea, applicable to any block of logic, not just object construction.

A well-named method also documents itself: `calculateAverage(scores)` tells the next reader what happens without them needing to read the loop inside it at all.

WATCH OUT FOR

Forgetting `return` in a method whose return type isn't `void` — this is a compile error, not a runtime surprise: Java insists every path through a non-void method actually returns a value.

Writing a method that's really two unrelated jobs squeezed into one — if you can't summarize what a method does in one short phrase, it may be doing too much.

QUICK RECAP

A method has a return type, a name, parameters, and a body — write the logic once, call it by name as often as you need.

`return` sends a value back to the caller and ends the method immediately, wherever it appears in the body.

Using Methods Multiple Times

What you actually get back for writing it once

`calculateAverage()` was written a single time in Section 8. Nothing stops it from being called five times, fifty times, or inside a loop — each call runs the exact same logic fresh, against whatever array it's handed that time.

```
int[] sectionA = {85, 90, 78, 92, 88};
int[] sectionB = {70, 88, 95, 80, 91};
int[] sectionC = {60, 75, 82, 91, 69};

System.out.println("Section A average: " + calculateAverage(sectionA));
System.out.println("Section B average: " + calculateAverage(sectionB));
System.out.println("Section C average: " + calculateAverage(sectionC));
```

PAUSE & THINK

Nothing here uses a loop to call `calculateAverage()` three times — the calls are just written out. If there were 40 sections instead of 3, what would you reach for instead, based on everything in Sections 4 and 6?

A for loop over an array of section arrays — exactly the pattern from Section 4, just calling a method instead of printing a value on each pass.

```
int[][] allSections = { sectionA, sectionB, sectionC };
for (int[] section : allSections) {
    System.out.println("Average: " + calculateAverage(section));
}
```

WHY THIS MATTERS

This is the entire payoff of writing a method in the first place: the cost of writing `calculateAverage()` was paid exactly once, and every call after that is free — one line, however many times you need it.

It also means a bug fix inside `calculateAverage()` automatically fixes every single place that calls it — compare that to hunting down five separate copy-pasted versions of the same loop.

QUICK RECAP

A method is defined once and can be called as many times as needed, from anywhere with access to it.

Calling a method repeatedly inside a loop is often more scalable than writing out each call by hand.

SECTION 10

Making Methods More Dynamic

The same method, doing something different every time

calculateAverage() already adapts to whatever array it's handed — that's parameters at work. But what if you need a method that decides its behavior based on more than just its input values — what if the exact same method name needs to accept different kinds or numbers of arguments altogether?

Parameters: The First Layer of Flexibility

```
public static String classifyScore(int score, int passingMark) {
    if (score >= passingMark) {
        return "Pass";
    }
    return "Fail";
}

classifyScore(72, 40); // "Pass" -- 72 clears a 40-mark bar
classifyScore(72, 75); // "Fail" -- the SAME score, against a different bar
```

The score didn't change between those two calls — the passing mark did. One method, two different “rules,” because the rule itself became a parameter instead of being hardcoded inside the method.

Method Overloading: Same Name, Different Signatures

Sometimes the most natural version of an idea genuinely needs a different number, or different types, of inputs. Java allows several methods to share the same name, as long as their parameter lists are different — called method overloading.

```
public static double calculateArea(double side) {           // square
    return side * side;
}
public static double calculateArea(double length, double width) { // rectangle
    return length * width;
}

calculateArea(5);           // 25.0 -- calls the one-parameter version
calculateArea(4, 6);        // 24.0 -- calls the two-parameter version
```

PAUSE & THINK

Java picks which calculateArea() to run based on how many arguments you pass, and what type they are — never based on the return type. Could you overload calculateArea(double side) with a second version that also takes just one double, but returns an int instead? Think about what Java would actually be able to tell those two versions apart by.

WHY THIS MATTERS

Parameters let one method body handle many different situations; overloading lets one method NAME cover several genuinely different but closely related operations, the way calculateArea() naturally means something slightly different for a square versus a rectangle.

You've already used overloaded methods constantly without necessarily naming the concept: println(String), println(int), and println(double) are all separate overloaded versions of the same method name.

WATCH OUT FOR

Trying to overload two methods that differ only in return type, with identical parameter lists — Java cannot tell them apart from a call site, so this fails to compile, even though it looks reasonable on paper.

QUICK RECAP

Parameters make a single method's behavior depend on its input values.

Overloading lets several methods share one name, distinguished only by their parameter lists — never by return type alone.

SECTION 11

Java Static Methods

Why main() never needs an object to run

Every Java program you've written starts execution at `public static void main(String[] args)` — and you've never once had to create an object before calling it. Section 5 of Week 4 introduced static fields and static blocks; this is the same keyword, applied to a method, and it explains exactly why `main()` works the way it does.

```
public class MathHelper {
    public static int square(int n) {    // static -- belongs to the CLASS
        return n * n;
    }

    public int cube(int n) {             // NOT static -- belongs to an OBJECT
        return n * n * n;
    }
}

// Calling them:
int result1 = MathHelper.square(5);     // called through the CLASS -- no object needed

MathHelper helper = new MathHelper();
int result2 = helper.cube(5);           // called through an OBJECT -- required for non-static
```

PAUSE & THINK

`square()` doesn't touch any field, doesn't need any per-object state at all — it just transforms whatever number it's handed. Given what you learned about static in Week 4, does that make `square()` a good or a bad candidate for being static? What would change if `MathHelper` had an instance field that `square()` needed to read?

A static method genuinely cannot reach instance fields or call this — the exact restriction from Week 4 — because a static method can be called before any object of that class exists at all. That's precisely why `main()` must be static: the JVM calls it to start your program, and at that moment, no object has been created yet for it to belong to.

WHY THIS MATTERS

Utility logic that doesn't depend on any object's individual state — like `square()`, or a unit conversion, or a formatting helper — is exactly what static methods are for: call them through the class name, with no object required.

This is also why `Math.sqrt()`, `Math.max()`, and `Math.random()` are all static — there's no meaningful “object” for square roots or randomness to belong to.

WATCH OUT FOR

Trying to call this or reach an instance field from inside a static method — the compiler rejects it outright, for the same reason a static method can run before any object exists: there may be no `this` to refer to at all.

QUICK RECAP

A static method belongs to the class itself and is called through the class name — no object required.

A static method cannot access instance fields or `this`, because it might run before any object of that class has ever been created.

Reach for static when a method's logic depends only on its parameters, never on a particular object's own state.

SECTION 12

Java Standard Library Methods

How much of this has someone already solved for you?

Sorting an array, searching it, converting it to a readable String for debugging — these are such common needs that Java ships working, tested, optimized versions of all of them, ready to call. Writing your own sort algorithm is a great learning exercise; reaching for one in real, working code is usually the wrong instinct.

```
import java.util.Arrays;

int[] scores = {85, 60, 92, 78, 90};

Arrays.sort(scores); // sorts IN PLACE -- modifies the original array
System.out.println(Arrays.toString(scores)); // [60, 78, 85, 90, 92] -- readable for debugging

int position = Arrays.binarySearch(scores, 85); // fast search -- ONLY valid on a sorted array
```

These live inside the Arrays class — a static-method-only utility class, exactly like the MathHelper idea from Section 11, except this one ships with Java itself. Math works the same way for numeric operations:

```
Math.max(12, 45); // 45
Math.min(12, 45); // 12
Math.sqrt(64); // 8.0
Math.abs(-17); // 17
Math.random(); // a double from 0.0 (inclusive) up to 1.0 (exclusive)
```

PAUSE & THINK

Arrays.sort(scores) has a void-like effect on scores itself — no assignment like scores = Arrays.sort(scores) is needed. Based on Section 5, can you explain why sort() is able to change the caller's original array without returning anything at all?

WHY THIS MATTERS

These methods are written, reviewed, and battle-tested by people who specialize in exactly this problem — Arrays.sort() uses a carefully tuned algorithm under the hood, not the simplest possible bubble sort a beginner might write by hand.

Recognising standard library methods on sight is also an interview and code-review skill: reinventing Arrays.sort() or Math.max() from scratch in real production code is a signal, not of thoroughness, but of not knowing the library exists.

WATCH OUT FOR

Assuming Arrays.sort() returns a new sorted array — it sorts in place and returns nothing at all (void); the original array itself is what changes, exactly like curveScores() in Section 5.

Calling Arrays.binarySearch() on an array that hasn't been sorted first — the method has no way to detect this, and will happily return a meaningless, incorrect result instead of an error.

QUICK RECAP

The Arrays class provides ready-made static methods for sorting, searching, and printing arrays — no need to write these from scratch.

The Math class provides static methods for common numeric operations — max, min, sqrt, abs, random, and more. Before writing a utility method yourself, it's worth checking whether Java's standard library already has exactly what you need.

WRAP-UP

Arrays and Methods: Why They Belong in the Same Document

One stores many values. The other packages logic for reuse. Real code needs both, together, constantly.

calculateAverage(int[] scores) from Section 8 only exists because Section 1 gave you a way to hand it many values through a single parameter. Arrays.sort(scores) from Section 12 only works the way it does because Section 5 already taught you that arrays are passed by reference. Every method in this document that takes an int[] parameter is a method and an array working together — that's not a coincidence of how this document happens to be organized, it's how almost all real Java code is actually structured: data shaped as arrays, flowing through methods that process it.

```
public static int[] curveAndReport(int[] scores, int bonus) {  
    int[] curved = new int[scores.length];           // SECTION 1: create a matching array  
    for (int i = 0; i < scores.length; i++) {        // SECTION 4/6: loop using .length  
        curved[i] = scores[i] + bonus;               // SECTION 3/5: access and modify by index  
    }  
    Arrays.sort(curved);                             // SECTION 12: standard library method  
    System.out.println("Curved: " + Arrays.toString(curved));  
    return curved;                                   // SECTION 8: returning a value  
}
```

QUICK RECAP

Arrays give your programs a way to hold structured, indexable collections of data — one value, a row of values, or a full grid.

Methods give your programs a way to name, reuse, and share logic — including logic that operates on those very arrays.

Almost every interesting Java program you'll write from here on is some combination of these two ideas, working on top of everything from Weeks 3–5: objects, encapsulation, and now, structured data flowing through reusable logic.

Cheat Sheet

Topic	Remember This
Creating an Array	new int[5] reserves a fixed number of boxes of one type — the size can never change afterward.
Initializing	new int[5] fills every box with that type's default (0 / false / null). A literal {1,2,3} fills and sizes it in one step.
Accessing	Indexing starts at 0; the last valid index is length - 1. Anything outside that range throws ArrayIndexOutOfBoundsException.
Accessing With a Loop	for (int i = 0; i < arr.length; i++) gives you the index; for (type x : arr) gives you just the values.
Modifying	Arrays are passed to methods by reference — a method can genuinely change the caller's original array.
length Property	array.length — no parentheses, it's a field. string.length() — parentheses, it's a method.
2D Arrays	int[][] is an array of arrays; grid.length is the row count, grid[row].length is that row's own length.

Topic	Remember This
User-Defined Methods	Return type + name + parameters + body, written once and called by name as often as needed.
Using Methods Repeatedly	The cost of writing a method is paid once; every call after that is a single line.
Dynamic Methods	Parameters adapt one method's behavior; overloading lets several methods share a name with different parameter lists.
Static Methods	Belong to the class, called without an object, and cannot access instance fields or this.
Standard Library Methods	Arrays.sort/toString/binarySearch and Math.max/min/sqrt/abs/random — check before you write it yourself.

Test Yourself

Answer from memory first. Whatever you cannot answer tells you exactly which section to reread.

1. Why does `int[] scores = new int[5]{1,2,3,4,5};` fail to compile, when both `new int[5]` and `{1,2,3,4,5}` are individually valid?
2. An array has 8 elements. What is the index of its first element, and what is the index of its last?
3. A method receives an `int[]` as a parameter and reassigns one of its elements. Does the caller see that change after the method returns? Why?
4. Explain, in one sentence, why `array.length` has no parentheses but `string.length()` does.
5. In a 2D array, what's the difference between `grid.length` and `grid[0].length`?
6. What's the difference between giving a method more parameters and giving it an overloaded version with a different parameter list?
7. Why must `main(String[] args)` be declared static?
8. Name two methods from the `Arrays` or `Math` class you'd reach for before writing your own version — and what each one saves you from writing.